

Kubernetes

Formation Avancée — Jour 3

Operators, CRDs, Admission Webhooks, API Aggregation et Scheduler Extensions

Étendre Kubernetes au-delà de ses limites natives

Module 13

Operator Pattern

Le problème que les Operators résolvent

Applications simples (stateless) : Kubernetes les gère nativement

- Deployment + Service + Ingress → suffisant

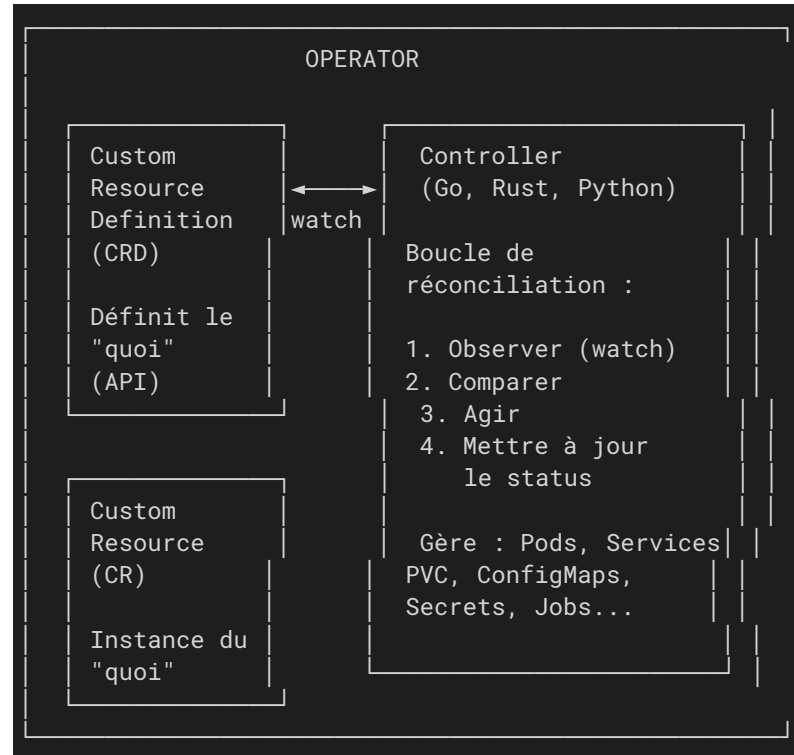
Applications complexes (stateful) : Kubernetes ne sait pas gérer seul

- Comment initialiser un cluster PostgreSQL avec réplication ?
- Comment faire un failover automatique si le primary tombe ?
- Comment gérer les backups, les mises à jour de schéma ?
- Comment monitorer la réplication lag ?

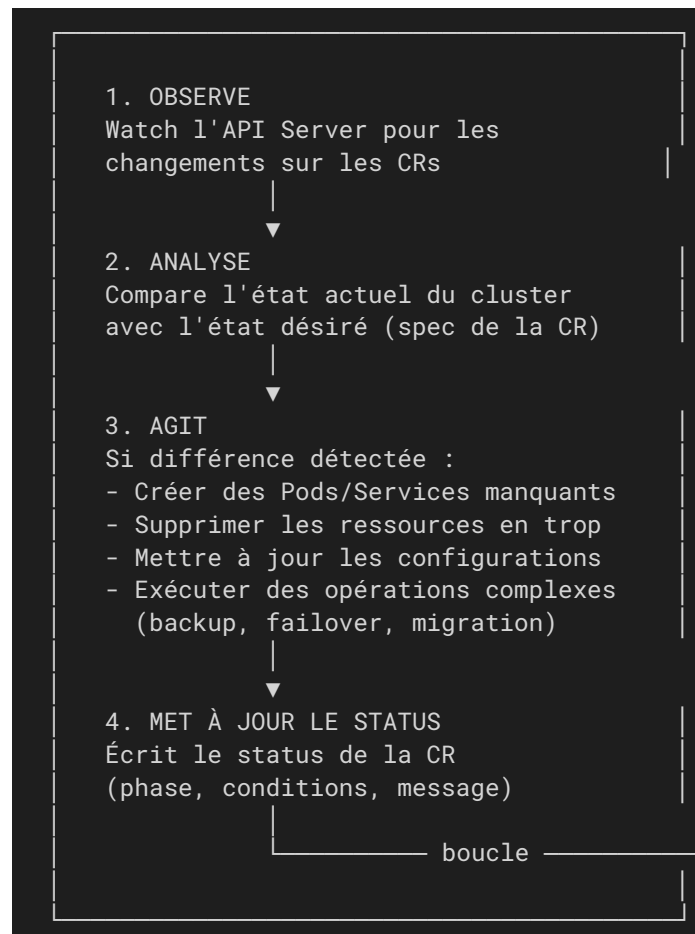
“ L'**Operator Pattern** encode le savoir-faire opérationnel d'un expert en code.

”

Architecture d'un Operator



Le cycle de réconciliation



Niveaux de maturité d'un Operator

Niveau	Capacité	Description	Exemple
1	Basic Install	Déploiement automatique	kubectl apply wrapper
2	Seamless Upgrades	Mises à jour gérées sans downtime	Rolling upgrade des pods
3	Full Lifecycle	Backup, restore, failover	pg_dump automatique, réplication
4	Deep Insights	Métriques, alertes, dashboards	Prometheus + Grafana intégrés
5	Auto Pilot	Auto-tuning, self-healing, auto-scaling	Ajustement automatique des configs

Frameworks pour créer des Operators :

- **Operator SDK** (Go, Ansible, Helm) – le plus utilisé
- **Kubebuilder** (Go) – scaffolding avancé
- **KUDO** – déclaratif
- **Metacontroller** – webhooks

Operators célèbres – Exemples concrets

PostgreSQL Operator (Zalando) :

```
apiVersion: acid.zalan.do/v1
kind: postgresql
metadata:
  name: mon-cluster-pg
spec:
  teamId: "formation"
  numberOfInstances: 3
  postgresql:
    version: "16"
  volume:
    size: 20Gi
  resources:
    requests:
      cpu: 500m
      memory: 1Gi
```

→ Crée automatiquement : 1 primary + 2 replicas, streaming replication, PVC, Services, secrets avec les credentials, patroni pour le failover

Operators célèbres — cert-manager

```
# ClusterIssuer Let's Encrypt
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: admin@example.com
    privateKeySecretRef:
      name: letsencrypt-prod-key
    solvers:
      - http01:
          ingress:
            class: nginx
---
# Certificat automatique
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: mon-app-tls
  namespace: production
spec:
  secretName: mon-app-tls-secret
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
  dnsNames:
    - app.example.com
```

Operators célèbres – Prometheus Operator

```
# ServiceMonitor – scraping automatique
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: mon-app-metrics
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: mon-app
  namespaceSelector:
    matchNames: ["production"]
  endpoints:
    - port: metrics
      path: /metrics
      interval: 30s
      scrapeTimeout: 10s
---
# PrometheusRule – alertes
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: mon-app-alerts
spec:
  groups:
    - name: mon-app
      rules:
        - alert: HighErrorRate
          expr: rate(http_errors_total[5m]) > 0.1
          for: 5m
          labels:
```

Module 14

Custom Resource Definitions (CRD)

Structure d'une CRD – Exemple complet

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: microservices.platform.utopios.net # <plural>.<group>
spec:
  group: platform.utopios.net # Groupe d'API
  scope: Namespaced # Namespaced | Cluster
  names:
    plural: microservices # kubectl get microservices
    singular: microservice # kubectl get microservice
    kind: Microservice # kind dans les YAML
    shortNames: ["ms"] # kubectl get ms
    categories: ["all"] # inclus dans kubectl get all
  versions:
    - name: v1
      served: true # Activée pour les requêtes API
      storage: true # Version stockée dans etcd
      schema:
        openAPIV3Schema:
          type: object
          required: ["spec"]
          properties:
            spec:
              type: object
              required: ["image", "replicas"]
              properties:
                image:
                  type: string
                  description: "Image Docker du microservice"
                replicas:
                  type: integer
```

CRD — Validation OpenAPI v3

```
properties:
  image:
    type: string
    pattern: "^[a-z0-9]+([._-][a-z0-9]+)*(/[a-z0-9]+([._-][a-z0-9]+)*)*(:[a-zA-Z0-9._-]+)?$"
  replicas:
    type: integer
    minimum: 1
    maximum: 20
    default: 1
  port:
    type: integer
    minimum: 1
    maximum: 65535
    default: 8080
  healthPath:
    type: string
    default: "/healthz"
  env:
    type: object
    additionalProperties:
      type: string
  resources:
    type: object
    properties:
      cpu:
        type: string
        pattern: "^[0-9]+m?$"
      memory:
        type: string
        pattern: "^[0-9]+(Mi|Gi)$"
```

CRD – Status, Printer Columns et Subresources

```
subresources:
  status: {}                # Active le sous-endpoint /status
  scale:                    # Active le sous-endpoint /scale (optionnel)
    specReplicasPath: .spec.replicas
    statusReplicasPath: .status.readyReplicas
additionalPrinterColumns:
- name: Image
  type: string
  jsonPath: .spec.image
- name: Replicas
  type: integer
  jsonPath: .spec.replicas
- name: Ready
  type: integer
  jsonPath: .status.readyReplicas
- name: Team
  type: string
  jsonPath: .spec.team
- name: Phase
  type: string
  jsonPath: .status.phase
- name: Age
```

CRD — Versions multiples et conversion

```
versions:
  - name: v1alpha1
    served: true
    storage: false          # Ancienne version, plus stockée
    schema:
      openAPIV3Schema:
        # ... schéma v1alpha1
  - name: v1
    served: true
    storage: true           # Version de stockage
    schema:
      openAPIV3Schema:
        # ... schéma v1 (avec nouveaux champs)
conversion:
  strategy: Webhook        # None | Webhook
  webhook:
    clientConfig:
      service:
        name: ms-conversion-webhook
        namespace: system
```

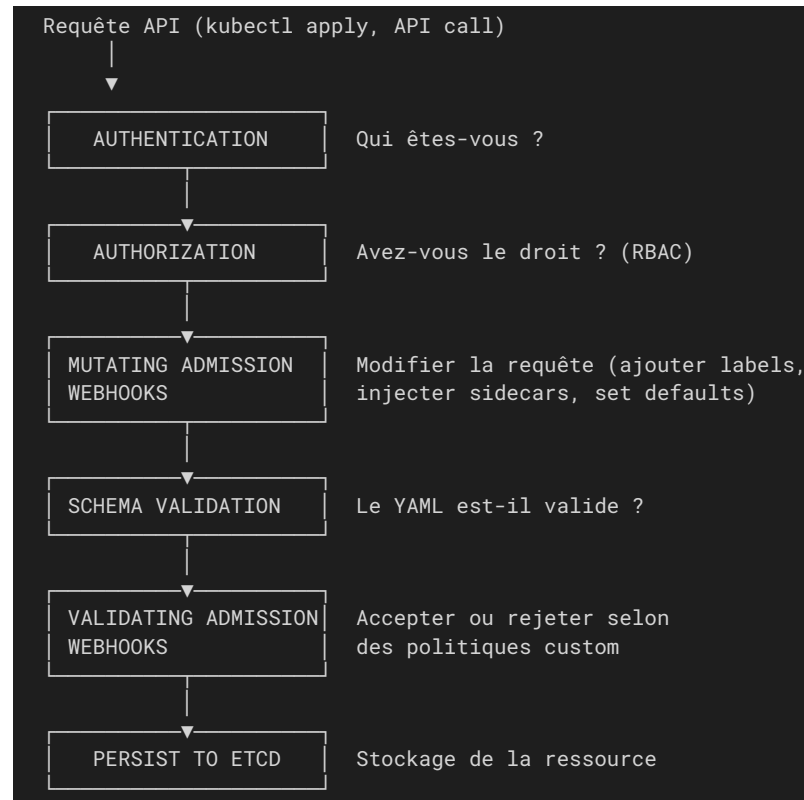
Custom Resource (CR) – Instance

```
apiVersion: platform.utopios.net/v1
kind: Microservice
metadata:
  name: api-gateway
  namespace: production
  labels:
    app: api-gateway
    managed-by: ms-operator
spec:
  image: "registry.corp.com/api-gateway:v2.3.1"
  replicas: 3
  port: 8080
  healthPath: "/actuator/health"
  team: platform
  env:
    SPRING_PROFILES_ACTIVE: "production"
    DB_HOST: "postgresql.production.svc.cluster.local"
    REDIS_HOST: "redis.production.svc.cluster.local"
  resources:
```

Module 15

Admission Controllers et Webhooks

Admission Controllers – Pipeline détaillé



Types d'Admission Controllers intégrés

Controller	Type	Rôle
NamespaceLifecycle	Validating	Empêche la création dans des ns en suppression
LimitRanger	Mutating	Applique les limites par défaut
ServiceAccount	Mutating	Injecte le SA par défaut
ResourceQuota	Validating	Vérifie les quotas
PodSecurity	Validating	Applique les Pod Security Standards
MutatingAdmissionWebhook	Mutating	Appelle vos webhooks custom
ValidatingAdmissionWebhook	Validating	Appelle vos webhooks custom

```
# Voir les admission controllers activés
kubectl exec -n kube-system kube-apiserver-formation-k8s-control-plane \
  -- kube-apiserver --help 2>&1 | grep enable-admission-plugins
```

Validating Webhook – Configuration complète

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: pod-security-policy
webhooks:
- name: pod-security.formation.utopios.net
  admissionReviewVersions: ["v1", "v1beta1"]
  sideEffects: None
  failurePolicy: Fail           # Fail | Ignore
  timeoutSeconds: 10
  clientConfig:
    service:
      name: webhook-server
      namespace: webhook-system
      path: "/validate-pods"
      port: 443
    caBundle: LS0tLS1CRUdJ...   # CA cert encodé en base64
  rules:
  - operations: ["CREATE", "UPDATE"]
    apiGroups: [""]
    apiVersions: ["v1"]
    resources: ["pods"]
    scope: "Namespaced"
  objectSelector:               # Optionnel : filtrer les objets
    matchExpressions:
    - key: skip-validation
      operator: DoesNotExist
```

Validating Webhook – Logique serveur

```
package main

import (
    "encoding/json"
    "fmt"
    "net/http"

    admissionv1 "k8s.io/api/admission/v1"
    corev1 "k8s.io/api/core/v1"
)

func handleValidate(w http.ResponseWriter, r *http.Request) {
    var review admissionv1.AdmissionReview
    json.NewDecoder(r.Body).Decode(&review)

    pod := corev1.Pod{}
    json.Unmarshal(review.Request.Object.Raw, &pod)

    allowed := true
    reasons := []string{}

    // Règle 1 : Pas de conteneur en root
    for _, c := range pod.Spec.Containers {
        if c.SecurityContext == nil ||
            c.SecurityContext.RunAsNonRoot == nil ||
            !*c.SecurityContext.RunAsNonRoot {
            allowed = false
            reasons = append(reasons,
                fmt.Sprintf("Container '%s': runAsNonRoot must be true", c.Name))
        }
    }

    // Règle 2 : Resources requests obligatoires
    for _, c := range pod.Spec.Containers {
        if c.Resources.Requests == nil {
            allowed = false
            reasons = append(reasons,
                fmt.Sprintf("Container '%s': resource requests required", c.Name))
        }
    }

    // Règle 3 : Labels obligatoires
    requiredLabels := []string{"app", "team"}
    for _, label := range requiredLabels {
        if _, ok := pod.Labels[label]; !ok {
            allowed = false
            reasons = append(reasons,
                fmt.Sprintf("Label '%s' is required", label))
        }
    }

    // Construire la réponse
    response := admissionv1.AdmissionReview{
        Response: &admissionv1.AdmissionResponse{
            UID:      review.Request.UID,
            Allowed:   allowed,
            Reasons:   reasons,
        },
    }

    if !allowed {
        response.Response.Result = &metav1.Status{

```

Mutating Webhook – Injection de sidecar

```
func handleMutate(w http.ResponseWriter, r *http.Request) {
    var review admissionv1.AdmissionReview
    json.NewDecoder(r.Body).Decode(&review)

    pod := corev1.Pod{}
    json.Unmarshal(review.Request.Object.Raw, &pod)

    // Vérifier si le sidecar est déjà présent
    for _, c := range pod.Spec.Containers {
        if c.Name == "istio-proxy" {
            // Déjà injecté, ne rien faire
            allowWithoutPatch(w, review)
            return
        }
    }

    // Construire le patch JSON
    patches := []map[string]interface{}{
        {
            "op": "add",
            "path": "/spec/containers/-",
            "value": map[string]interface{}{
                "name": "istio-proxy",
                "image": "istio/proxyv2:1.20",
                "ports": []map[string]int{
                    {"containerPort": 15001},
                },
                "resources": map[string]interface{}{
                    "requests": map[string]string{
                        "cpu": "100m", "memory": "128Mi",
                    },
                },
            },
        },
        {
            "op": "add",
            "path": "/metadata/labels/sidecar.istio.io~1inject",
            "value": "true",
        },
    }

    patchBytes, _ := json.Marshal(patches)
    patchType := admissionv1.PatchType_JSONPatch
}
```

Cas d'usage des webhooks

Validating Webhooks :

Cas d'usage	Description
Sécurité	Interdire les conteneurs root, images non signées
Conformité	Exiger des labels (team, cost-center), limiter les registres
Quotas custom	Limiter le nombre de PVC par namespace
Naming conventions	Vérifier le format des noms de ressources
Politique réseau	Exiger une NetworkPolicy avant de créer des pods

Cas d'usage des webhooks

Mutating Webhooks :

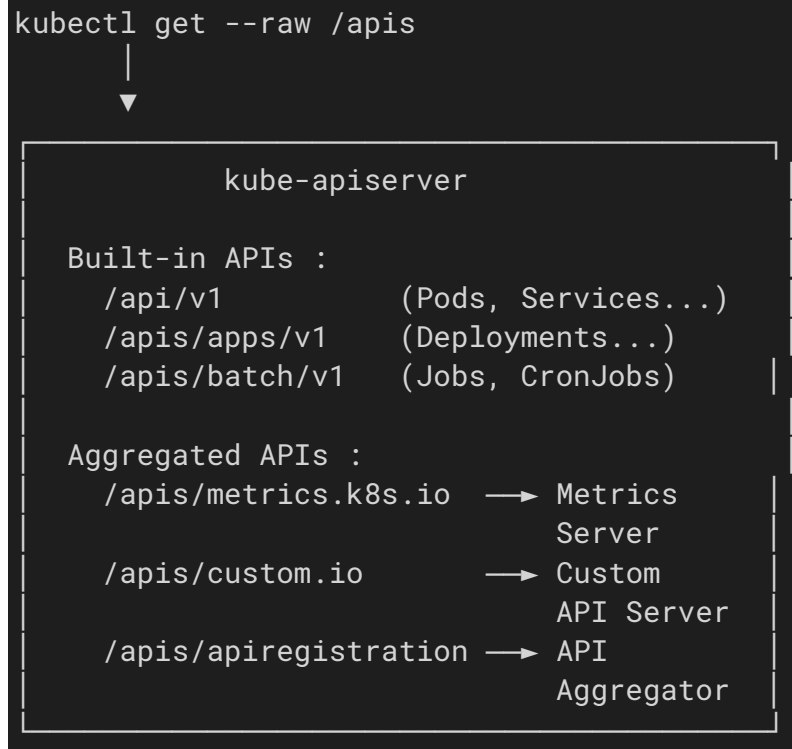
Cas d'usage	Description
Sidecar injection	Istio, Linkerd, Vault Agent
Labels automatiques	Ajouter team, env, git-commit
Default values	Resources requests, imagePullPolicy
Image rewriting	Rediriger vers un mirror de registre interne
Annotation enrichment	Ajouter des annotations Prometheus

Module 16

API Aggregation et Extension

API Aggregation Layer

L'**API Aggregation** permet d'ajouter de nouvelles APIs au kube-apiserver :



APIService – Enregistrement

```
apiVersion: apiregistration.k8s.io/v1
kind: APIService
metadata:
  name: v1beta1.metrics.k8s.io
spec:
  service:
    name: metrics-server
    namespace: kube-system
  group: metrics.k8s.io
  version: v1beta1
  insecureSkipTLSVerify: true
  groupPriorityMinimum: 100
  versionPriority: 100
```

APIService — Enregistrement

```
# Voir les API Services
kubectl get apiservices
kubectl get apiservices v1beta1.metrics.k8s.io -o yaml

# Le Metrics Server (installé par notre script) fournit :
kubectl top nodes
kubectl top pods -n dev
kubectl top pods --sort-by=cpu
kubectl top pods --sort-by=memory
```

Custom Metrics pour HPA avancé

```
# HPA avec métriques custom (nécessite un adaptateur comme Prometheus Adapter)
```

```
apiVersion: autoscaling/v2
```

```
kind: HorizontalPodAutoscaler
```

```
metadata:
```

```
  name: mon-app-custom-hpa
```

```
spec:
```

```
  scaleTargetRef:
```

```
    apiVersion: apps/v1
```

```
    kind: Deployment
```

```
    name: mon-app
```

```
  minReplicas: 2
```

```
  maxReplicas: 50
```

```
  metrics:
```

```
    # Métrique CPU standard
```

```
  - type: Resource
```

```
    resource:
```

```
      name: cpu
```

```
    target:
```

```
      type: Utilization
```

```
      averageUtilization: 70
```

```
    # Métrique custom (requêtes par seconde)
```

```
  - type: Pods
```

```
    pods:
```

```
      metric:
```

```
        name: http_requests_per_second
```

```
      target:
```

```
        type: AverageValue
```

```
        averageValue: "500"
```

```
    # Métrique externe (longueur de queue)
```

```
  - type: Object
```

```
    object:
```

```
      metric:
```

```
        name: queue_messages_ready
```

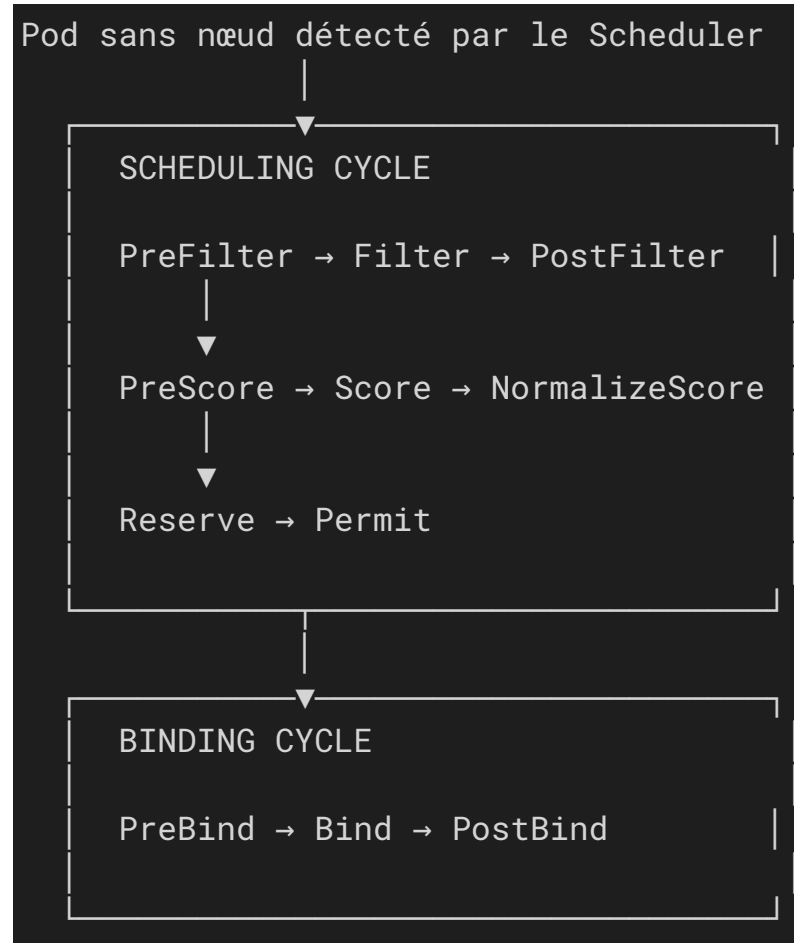
```
      describedObject:
```

```
        apiVersion: v1
```

Module 17

Scheduler Extensions

Scheduling Framework – Phases détaillées



Node Affinity – Règles de placement

```
spec:
  affinity:
    nodeAffinity:
      # OBLIGATOIRE : le pod DOIT aller sur ces nœuds
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: zone
                operator: In
                values: ["zone-a", "zone-b"]
              - key: node-type
                operator: NotIn
                values: ["gpu"]
          # PRÉFÉRENCE : le pod DEVRAIT aller sur ces nœuds (si possible)
        preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 80                # Poids de 1 à 100
            preference:
              matchExpressions:
                - key: disk-type
                  operator: In
                  values: ["ssd"]
          - weight: 20
            preference:
              matchExpressions:
```

Pod Affinity et Anti-Affinity

```
spec:
  affinity:
    # Pod Affinity : placer AVEC d'autres pods
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values: ["redis"]
            topologyKey: kubernetes.io/hostname
          # → Le pod DOIT être sur le même nœud qu'un pod redis

    # Pod Anti-Affinity : placer LOIN d'autres pods
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values: ["web"]
```

Pod Affinity et Anti-Affinity

“ L'anti-affinité est essentielle pour la **haute disponibilité** : répartir les réplicas sur différents nœuds. ”

Taints et Tolerations

```
# Ajouter un taint à un nœud
kubectl taint nodes worker-1 dedicated=gpu:NoSchedule
kubectl taint nodes worker-2 maintenance=true:NoExecute

# Supprimer un taint
kubectl taint nodes worker-1 dedicated=gpu:NoSchedule-
```

Taints et Tolerations

Effect	Comportement
NoSchedule	Nouveaux pods non tolérés ne sont PAS schedulés
PreferNoSchedule	Essaie d'éviter mais peut scheduler si nécessaire
NoExecute	Pods existants non tolérés sont évictés

```
# Pod qui tolère le taint GPU
spec:
  tolerations:
    - key: "dedicated"
      operator: "Equal"
      value: "gpu"
      effect: "NoSchedule"
    - key: "maintenance"
      operator: "Exists"           # Tolère toute valeur
      effect: "NoExecute"
      tolerationSeconds: 3600     # Tolérer pendant 1h puis éviction
```

Topology Spread Constraints

Distribuer les pods uniformément à travers les zones/nœuds :

```
spec:
  topologySpreadConstraints:
    - maxSkew: 1                                # Écart max entre zones
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule          # DoNotSchedule | ScheduleAnyway
      labelSelector:
        matchLabels:
          app: web
    - maxSkew: 1
      topologyKey: kubernetes.io/hostname
      whenUnsatisfiable: ScheduleAnyway         # Soft constraint
      labelSelector:
        matchLabels:
          app: web
```

Topology Spread Constraints

Résultat avec 6 répliques sur 3 zones :

zone-a: [pod-1] [pod-2]

zone-b: [pod-3] [pod-4]

zone-c: [pod-5] [pod-6]

maxSkew=1 garantit : max 2 pods par zone ($6/3=2$, skew max 1)

Custom Scheduler

```
# Utiliser un scheduler personnalisé pour certains pods
apiVersion: v1
kind: Pod
metadata:
  name: gpu-workload
spec:
  schedulerName: gpu-scheduler      # Au lieu de default-scheduler
  containers:
    - name: training
      image: tensorflow/tensorflow:latest-gpu
      resources:
        limits:
          nvidia.com/gpu: 1
```

Custom Scheduler

Cas d'usage pour Custom Schedulers :

- **GPU-aware scheduling** : placement selon les types de GPU disponibles
- **Network-aware scheduling** : placement selon la latence réseau
- **Cost-aware scheduling** : placement sur les nœuds les moins chers (cloud)
- **Data locality** : placement près des données (HDFS, Ceph)
- **Bin-packing** : remplir les nœuds au maximum avant d'en utiliser un nouveau
- **Multi-cluster** : scheduling fédéré entre clusters (Admiralty, Ligo)